Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

第八章 Flask-Web数据库 回顾

- 数据库 基本概念
- Flask-SQLAlchemy ORM、简介
- 基本实现 导入配置、初始化、构建模型类、数据库基本操作
- 在视图函数中操作数据库 数据库操作
- 关系建模 一对多、多对对
- 数据库操作扩展 多种查询方式
- 数据库迁移 flask-migrate的使用

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

第九章 大型应用的结构及蓝图



成都信息工程大學
Chengdu University of Information Technology

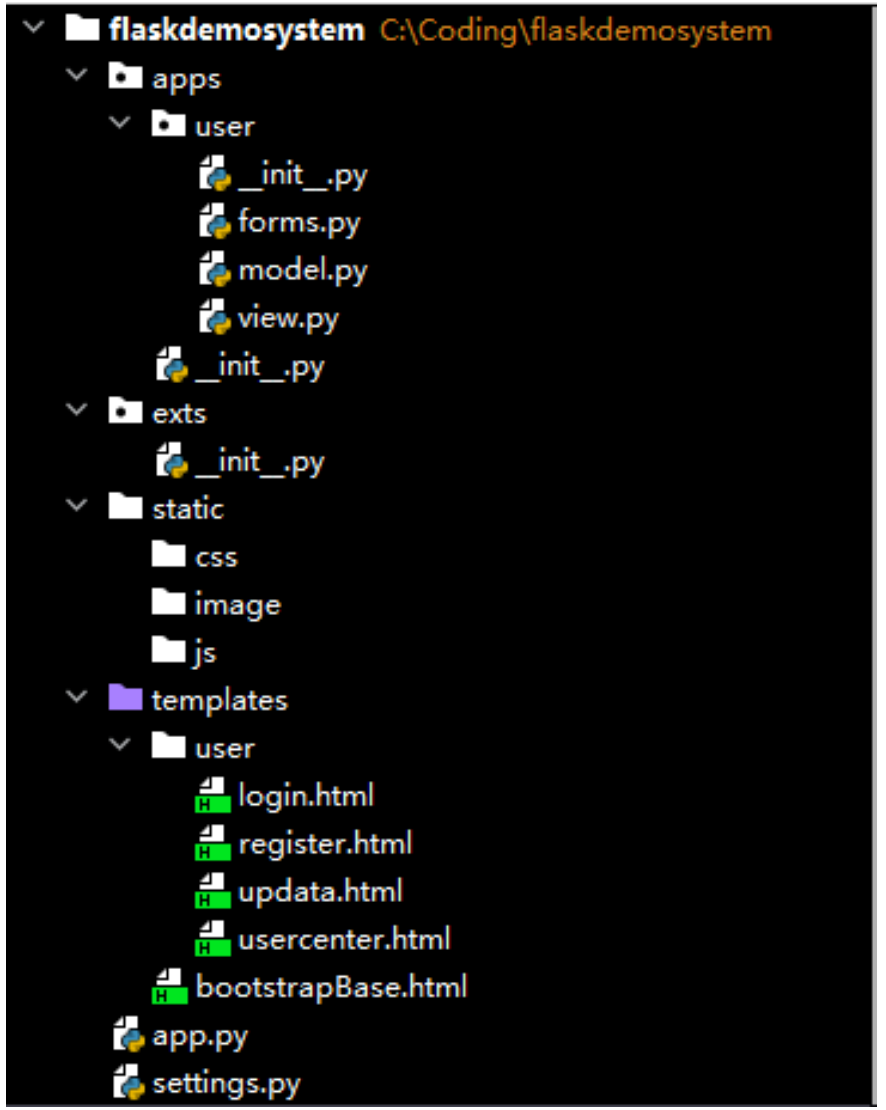
- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

大型应用的基本结构

- 结构化系统文件的必要性
  - 对于小型的WEB应用，使用单一的脚本文件方便、快捷，阅读方便，但是其伸缩性不好，当应用的功能变多、应用变复杂后，使用单个大型源码文件会导致很多问题，主要体现在可维护性上
  - 有必要对大型应用采用相应的结构化文件组织，方便后期维护管理
- 一般的组织结构如下：
  - 启动文件命名为app.py 置于项目根目录下
  - 配置文件命名为settings.py/config.py置于项目根目录下
  - 扩展实例置于exts包下的__init__.py中，exts包置于项目根目录下
  - 工厂函数置于apps包的__init__.py中， apps包置于项目根目录下

大型应用的基本结构

- 一般的组织结构如下：
  - 各模块内容，按照模块分别创建文件夹置于apps包，模块文件夹内分别将各模块的模型(model.py)、视图(view.py)、表单(forms.py)等相关内容
  - 模板文件置于templates中，将各模块的模板分别放置在以模块命名的文件夹内
  - 静态文件按照css、js、image创建文件夹，置于static文件夹下



第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

启动文件

- 整个系统的启动入口
  - 一般命名为app.py，其他名称亦可
  - 包含用工厂函数配置好的Flask实例对象
  - 系统的启动指令app.run()

```
1  from apps import create_app
2
3  app = create_app()
4
5  if __name__ == '__main__':
6      app.run(debug=True)
7
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

配置文件

- 整个系统的配置项文件
  - 单独将整个系统的配置项和参数写在配置文件中，命名为settings.py或config.py
  - 可以是以字典形式的配置项；也可以将配置内容写在一个config类中，用类中属性值来设置属性，并支持不同环境配置的继承
  - 在工厂函数中，利用app.config.from_object()或其他函数实现配置项与项目链接
  - 为了让配置更灵活安全，可以将通用配置设置为父类，专属配置设置在子类中进行设置

```
from_object(self, obj)                          Config
from_envvar(self, variable_name, silent)        Config
fromkeys(cls, __iterable, __value)                dict
from_file(self, filename, load, silent, text)   Config
from_pyfile(self, filename, silent)             Config
from_mapping(self, mapping, kwargs)             Config
from_prefixed_env(self, prefix, loads)          Config
```

```
1  ENV = 'development'
2  DEBUG = True
3  SECRECT_KEY = "a random string secrect key"
4  SQLALCHEMY_DATABASE_URI = "mysql+pymysql://root:123456@127.0.0.1:3306/flaskdemosystem"
5  SQLALCHEMY_TRACK_MODIFICATIONS = False
```

```
class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or 'hard to guess string'
    MAIL_SERVER = os.environ.get('MAIL_SERVER', 'smtp.googlemail.com')
    MAIL_PORT = int(os.environ.get('MAIL_PORT', '587'))
    MAIL_USE_TLS = os.environ.get('MAIL_USE_TLS', 'true').lower() in \
        ['true', 'on', '1']
    MAIL_USERNAME = os.environ.get('MAIL_USERNAME')
    MAIL_PASSWORD = os.environ.get('MAIL_PASSWORD')
    FLASKY_MAIL_SUBJECT_PREFIX = '[Flasky]'
    FLASKY_MAIL_SENDER = 'Flasky Admin <flasky@example.com>'
    FLASKY_ADMIN = os.environ.get('FLASKY_ADMIN')
    SQLALCHEMY_TRACK_MODIFICATIONS = False

    @staticmethod
    def init_app(app):
        pass

class DevelopmentConfig(Config):
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.environ.get('DEV_DATABASE_URL') or \
        'sqlite:///' + os.path.join(basedir, 'data-dev.sqlite')

class TestingConfig(Config):
    TESTING = True
    SQLALCHEMY_DATABASE_URI = os.environ.get('TEST_DATABASE_URL') or \
        'sqlite://'

class ProductionConfig(Config):
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or \
        'sqlite:///' + os.path.join(basedir, 'data.sqlite')
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

扩展实例

- 开发项目时我们必然会用到许多的Flask扩展，这些扩展都需要在使用前进行实例化
  - 一般将flask扩展的实例化放到exts包中进行
  - 具体实例在exts包的__init__.py文件中，来实现各种扩展的实例化

```python
from flask_sqlalchemy import SQLAlchemy
from flask_bootstrap import Bootstrap
from flask_wtf import CSRFProtect

# 创建一个数据库与app的映射对象
db = SQLAlchemy()
# 创建bootstrap实例
bootstrap = Bootstrap()
# 创建csrf实例
csrf = CSRFProtect()
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

工厂函数

- 工厂函数的必要性
  - 在单个文件中开发应用是很方便，但应用在全局作用域中创建，无法动态修改配置。运行脚本时，应用实例已经创建，再修改配置为时已晚，因此我们需要延迟创建应用实例，把创建过程移到可显示调用的工厂函数中。
- 工厂函数的内容
  - 一般将工厂函数放在项目包的__init__.py文件中，使用create_app()命名
  - 工厂函数一般包含以下内容：
    - ✓ 创建flask实例app，app = Flask(__name__)
    - ✓ 实现对app的实际配置，app.config.from_object()
    - ✓ 实现对flask扩展与app的连接
    - ✓ 注册蓝图app.register_blueprint()
    - ✓ 返回配置好的app

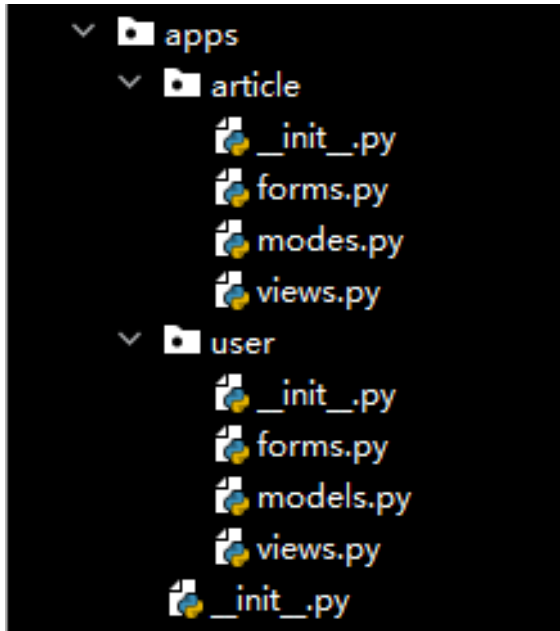
```
7   def create_app():
8       app = Flask(__name__,template_folder='../templates',static_folder='../static')
9       app.config.from_object(settings)    # 加载配置
10      app.config['SECRET_KEY'] = 'KDSAKFHKASLKLASDKLASJKDNAJND'
11      db.init_app(app)                    # 连接数据库
12      bootstrap.init_app(app)             # 连接bootstrap
13      csrf.init_app(app)                  # 添加csrf保护
14      # 注册用户蓝图
15      app.register_blueprint(user_bp)     # 将用户蓝图对象绑定到app
16      return app
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

包/模块/蓝图

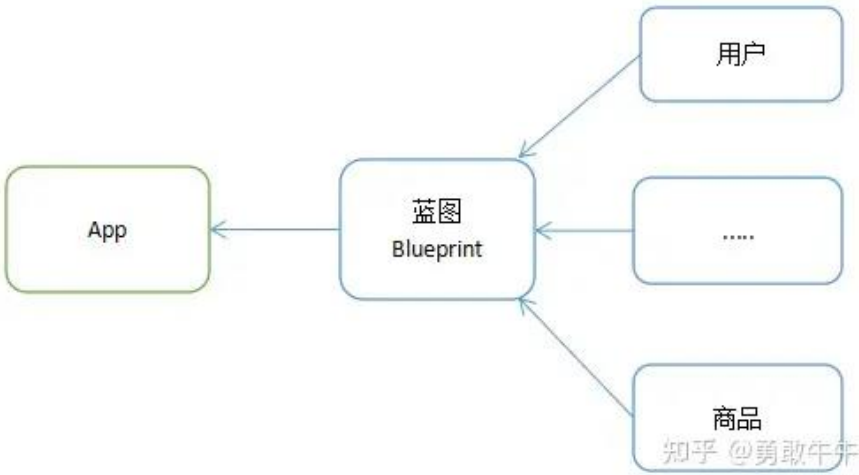
- 包
  - 包用于存放项目的所有代码(有时也将模板、静态文件放在包内)，一般将项目的包命名为apps
  - apps中按照系统设计的模块，分别创建子包存放模块的代码
  - apps中的__init__.py为包的初始化文件，一般将项目的工厂函数写在其中
- 模块
  - 依据系统的设计，按照功能类别或按照处理的对象不同，将系统分为不同的模块，每个模块中的功能有一定的相关性
  - 系统的子模块以子包的形式存放在apps包中，并将每个模块的模型(models.py)、视图(views.py)和表单(forms)分别用不同的文件来管理



包/模块/蓝图

蓝图

- 转换成应用工厂函数的操作让定义路由变复杂了，在单脚本应用中，应用实例存在于全局作用域中，路由直接由app.route()定义
- 使用应用组织结构化后，应用(app)在运行的时候才创建，只有调用 create_app() 之后才能使用 app.route 装饰器，这时定义路由就太晚了
- Flask使用蓝图(blueprint)来解决以上问题，Blueprint 是一种组织一组相关视图及其他代码的方式
- 蓝图和应用类似，可以定义路由或错误处理程序，其不同之处在于，蓝图定义的路由和错误处理处于休眠状态，只有将蓝图注册到app上之后，它们才是应用的一部分



知乎 @勇敢牛牛

包/模块/蓝图

蓝图

- 蓝图通过实例化一个Blueprint类对象创建，包含两个必须的参数，蓝图名和所在的包或模块(__name__)
- 一般在在子包的__init__.py中实例化蓝图，并在使用蓝图实例的模块文件中引入

```
1  from flask import Blueprint
2  #实例化蓝图，定义用户蓝图/模块，其中user为蓝图，__name__是本文件的名称
3  user_bp = Blueprint( name: 'user', __name__)
4  #引入需要使用蓝图的模块文件，并在使用蓝图的模块中引入蓝图实例
5  from . import views
```

```
from . import user_bp
```

- 蓝图在工厂函数中注册到应用中，app.register_blueprint()，若有多个蓝图均在工厂函数中逐一注册，一般一个模块一个蓝图

```
3  from apps.user.views import user_bp
```

```
15     app.register_blueprint(user_bp)      # 将用户蓝图对象绑定到app
```

包/模块/蓝图

蓝图

- 利用蓝图定义路由：@蓝图名.route(‘路由’,methods=[‘POST’，‘GET’],endpoint=‘’)

  建议在模块中定义路由时
  均加上endpoint，方便后续
  反向构建路由的URL

```python
@user_bp.route('/')
def user_center():...

# 注册视图
@user_bp.route('/register',methods=['POST','GET'],endpoint='register')
def register():...

# 登录视图
@user_bp.route('/login',methods=['POST','GET'])
def login():...

# 退出视图
@user_bp.route('/logout',methods=['POST','GET'])
def logout():...

# 更新视图
@user_bp.route('/update',methods=['POST','GET'])
def update():...
```

- 利用url_for()反向构建路由的URL：url_for(‘蓝图名.endpoint’)

```
<form method="POST" action="{{ url_for('user.register')}}">
    <div class="form-group">
    {{ wtf.quick_form(userform) }}
    </div>
</form>
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

需求文件

需求文件

- 用于记录所有依赖包及其版本号，一般命名为requirements.txt置于项目根目录下
- 一般在一台新计算机上新生成虚拟环境时需要，例如系统部署时所使用的设备
- 在终端使用pip freeze > requirements.txt 生成需求文件
- 创建虚拟环境副本:
  - ✓ 首先创建一个虚拟环境
  - ✓ 然后在其中运行 pip install –r requirements.txt
  - ✓ 则实现环境的复制

```
alembic==1.12.0
blinker==1.6.2
cffi==1.16.0
click==8.1.7
colorama==0.4.6
cryptography==41.0.5
dnspython==2.4.2
dominate==2.8.0
email-validator==2.1.0.post1
Flask==3.0.0
Flask-Bootstrap==3.3.7.1
Flask-Migrate==4.0.5
Flask-SQLAlchemy==3.1.1
Flask-WTF==1.2.1
greenlet==3.0.0
idna==3.4
importlib-metadata==6.8.0
itsdangerous==2.1.2
Jinja2==3.1.2
Mako==1.2.4
MarkupSafe==2.1.3
pycparser==2.21
PyMySQL==1.1.0
SQLAlchemy==2.0.22
typing_extensions==4.8.0
visitor==0.1.3
Werkzeug==3.0.0
WTForms==3.0.1
```

第九章 大型应用的结构及蓝图

- 大型应用的基本结构
- 启动文件
- 配置文件
- 扩展实例
- 工厂函数
- 包/模块/蓝图
- 需求文件
- 系统分析和设计简述

系统分析和设计简述

需求分析

https://zhuanlan.zhihu.com/p/600301253

- 是开发人员经过深入细致的调研和分析，准确理解用户和项目的功能、性能、可靠性等具体要求，将用户非形式的需求表述转化为完整的需求定义，从而确定系统必须做什么的过程
- 其核心就是明确：用户的真实目的；产品的解决方案
- 需求分析的步骤：
  - ✓ **需求转化**，将收集来的需求整理到需求分析记录表里面，确定需求清单
  - ✓ **确定需求属性**，明确需求属于新增功能、功能改进、体验提升、Bug修复、内部需求，并确认需求是必要型需求、期望型需求、兴奋型需求、无差别需求还是逆向需求
  - ✓ **判断需求真伪**，深度挖掘需求背后的动机，从角色、场景和流程三个角度进行分析
  - ✓ **分析需求价值**，从用户需求价值和产品方案价值两个 角度来分析
  - ✓ **评估实现难度，挖掘价值最大化的需求**，性价比最大的需求优先完成

系统分析和设计简述

需求分析

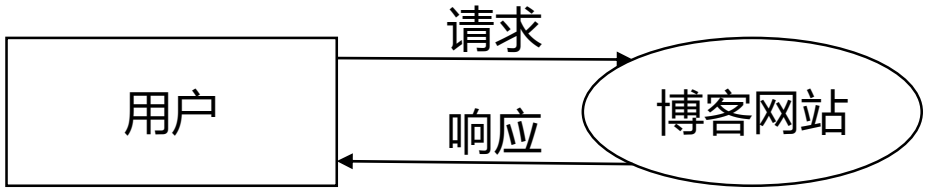
博客网站系统的需求清单

1. 用户注册和登录功能：
   - 允许用户注册账号，管理个人资料
   - 提供登录功能，让已注册用户方便地登录、身份认证
   - 提供退出功能，实现用户的退出
2. 博客文章发布和管理：
   - 支持用户发布博客文章，包括文字内容、图片、视频等
   - 提供富文本编辑器，方便用户编辑和格式化博客文章的内容
   - 支持文章的标签分类、标签云和搜索功能，方便用户组织和浏览文章
   - 允许用户保存博客文章为草稿，并进行多次编辑和修改
   - 允许查看和编辑自己的博客
3. 评论和互动功能：
   - 允许读者对博客文章发表评论和留言，与作者进行互动
   - 支持对评论进行回复和点赞，增加用户间的交流和互动性
   - 提供评论管理功能，让博主可以审查和管理评论内容

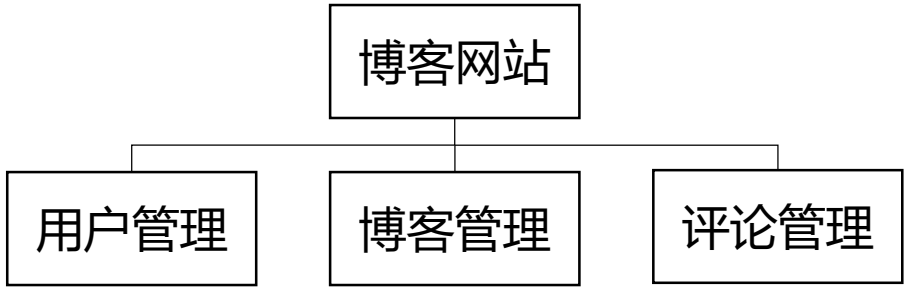
系统分析和设计简述

系统分析和概要设计

- 依据需求清单对系统进行结构化/面向对象分析，得出系统功能结构图和数据库相关设计(数据库选型、数据表、表之间的关系)
- 结构化分析一般采用数据流图对系统进行分析



- 依据分析结果得出系统功能结构图



用户管理：用户注册、用户登录、用户退出、资料编辑
博客管理：发表博客、编辑博客、分类分类、显示博客
评论管理：发表评论、编辑评论、管理评论

系统分析和设计简述

系统分析和概要设计

- UML 是在开发阶段，说明、可视化、构建和书写一个面向对象软件密集系统的制品的开放方法，大多以图表的方式表现出来
  - ✓ 类图：描述类的特性以及类之间的静态关系
  - ✓ 顺序图：描述参与者之间的动态调用关系
  - ✓ 组件图：描述物理组件静态关系
  - ✓ 部署图：定义系统中软硬件的物理体系结构，部署的服务器数量、关键组件的部署位置
  - ✓ 用例图：从用户角度描述系统功能，分为业务用例和系统用例
  - ✓ 活动图：描述了业务实现用例的工作流程
  - ✓ 对象图：系统中的多个对象在某一时刻的状态
  - ✓ 状态图：是描述状态到状态控制流，常用于动态特性建模
  - ✓ 协作图：描述对象之间的协助关系
  - ✓ 包图：对构成系统的模型元素进行分组整理的图
  - ✓ 组合结构图：表示类或者构建内部结构的图
  - ✓ 交互概览图：用活动图来表示多个交互之间的控制关系的图

系统分析和设计简述

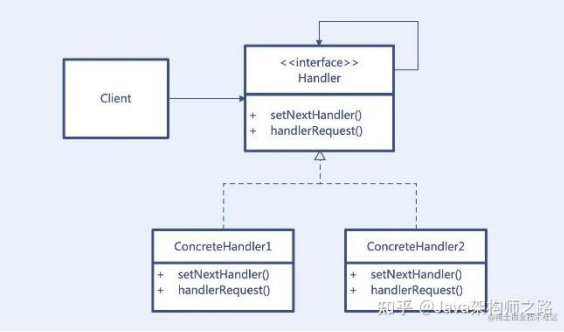
成都信息工程大學 Chengdu University of Information Technology

系统分析和概要设计

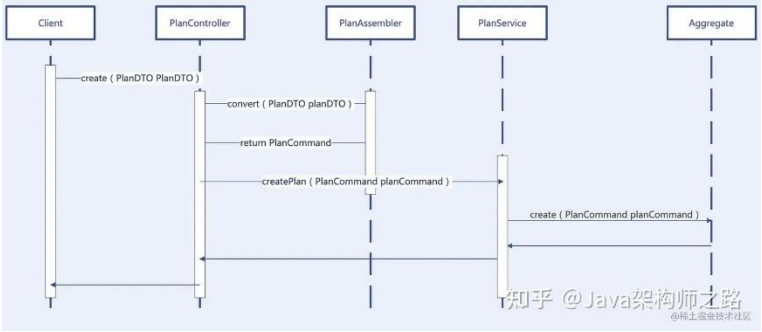
https://www.icourse163.org/course/PKU-1003177002

https://zhuanlan.zhihu.com/p/622532437

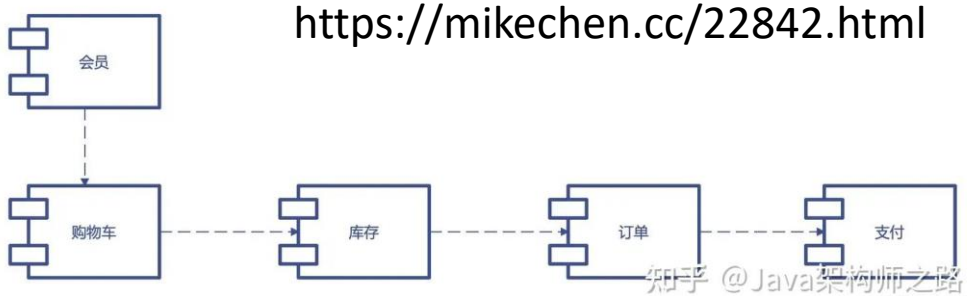
https://mikechen.cc/22842.html



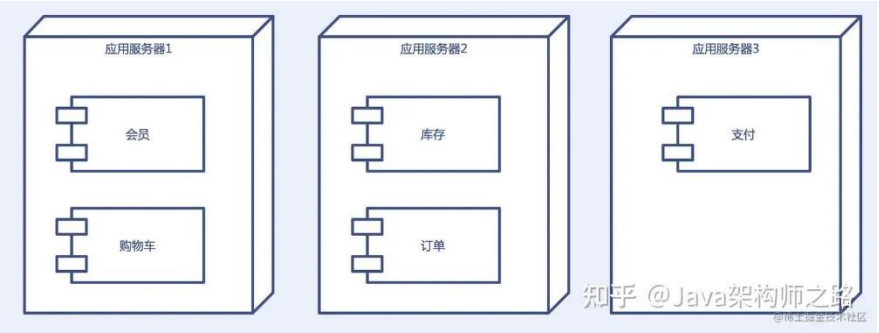
类图



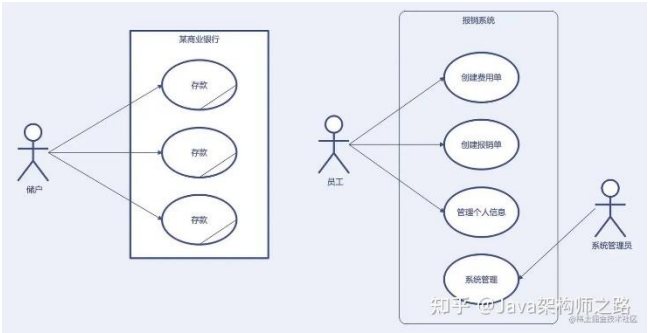
序列/顺序图



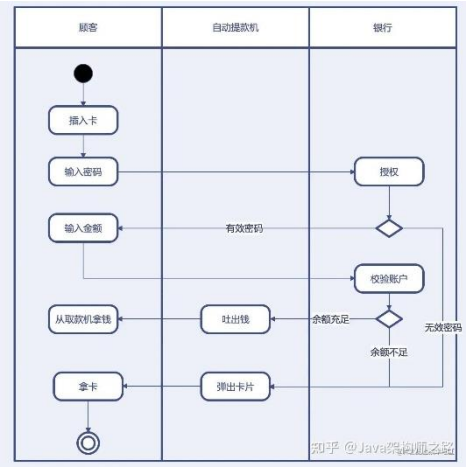
组件图



部署图



用例图

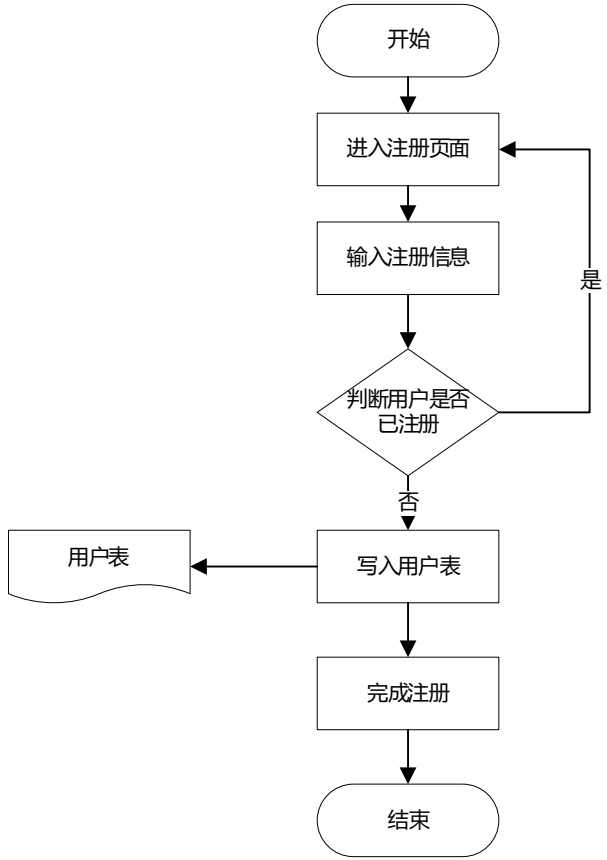


活动图

系统分析和设计简述

详细设计

- 对每个功能的具体实现流程进行详细的设计，一般使用程序流程图来实现
- 每个功能必须有明确开始、结束、
  输入(用户注册信息，用户名、密码，个人信息等)、
  输出(更新后的用户表)、
  加工处理(将用户信息写入用户表)
  判断(判断待注册的用户是否已经在系统存在)



第九章 大型应用的结构及蓝图 总结

- 大型应用的基本结构　　包含内容，如何组织
- 启动文件　　创建app实例，运行项目，app.py
- 配置文件　　配置字典，settings.py
- 扩展实例　　辅助功能实例化
- 工厂函数　　实例化前的app配置，create_app()
- 包/模块/蓝图　　功能代码组织、蓝图定义和注册 Blueprint
- 需求文件　　所需依赖，requirements.txt
- 系统分析和设计简述　　系统分析设计过程

第二次 课后作业

1. 依据相关资料，学习理解软件设计相关图形(功能结构图、业务流程图、类图、序列图、活动图和用例图等)的表达及画法，并简述大型应用系统分析、设计和实现的流程、方法和技术

- 要求：
  - 独立完成，自己组织语言描述，不少于1000字
  - 12月6日前提交
  - 手写稿，注明“学号+姓名”，如：“2020083024林兆君”